MECHATRONICS AND IOT ASSIGNMENT REPORT - 9

IoT-Based Smart Refrigerator Monitoring System

PROJECT REPORT

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S 24MER024

LOGESH S 24MER027

1. Project Overview:

The Smart Refrigerator Monitoring System is an IoT-based solution developed to continuously monitor the internal temperature and humidity conditions of a refrigerator. Maintaining proper environmental conditions inside a refrigerator is essential for preserving food quality, preventing spoilage, and ensuring efficient operation of the cooling system. Traditional methods of manually checking refrigerator conditions do not provide continuous monitoring and may fail to detect sudden changes that can affect stored food items.

The system continuously measures temperature and humidity using a sensor and transmits the collected data to a cloud platform through Wi-Fi. This enables users to remotely monitor refrigerator conditions in real time and analyze historical data to identify abnormal environmental changes. By providing continuous monitoring and remote access to sensor information, the system helps improve food safety, supports preventive maintenance, and demonstrates the application of IoT technology in smart home and food-storage environments.

2. Project Statement:

Food stored in refrigerators requires suitable temperature and humidity conditions for proper preservation. Manual checking of refrigerator conditions does not provide continuous monitoring and may fail to detect sudden changes. Such variations can affect food quality, increase the risk of spoilage, and reduce the overall efficiency of the refrigeration system.

Therefore, an IoT-based Smart Refrigerator Monitoring System is proposed to continuously monitor the internal environmental conditions of a refrigerator and provide remote access to the collected data through a cloud platform. The system is designed to:

- Continuously measure refrigerator temperature.
- Continuously measure refrigerator humidity.
- Monitor environmental conditions in real time.
- Detect abnormal temperature and humidity variations.
- Transmit sensor data through Wi-Fi.
- Store sensor readings on a cloud platform.
- Enable remote monitoring through a dashboard.

- Support historical data analysis for performance evaluation.

The proposed system helps users monitor refrigerator conditions remotely, identify potential refrigeration problems at an early stage, and maintain suitable storage conditions for food preservation.

3. Proposed Solution:

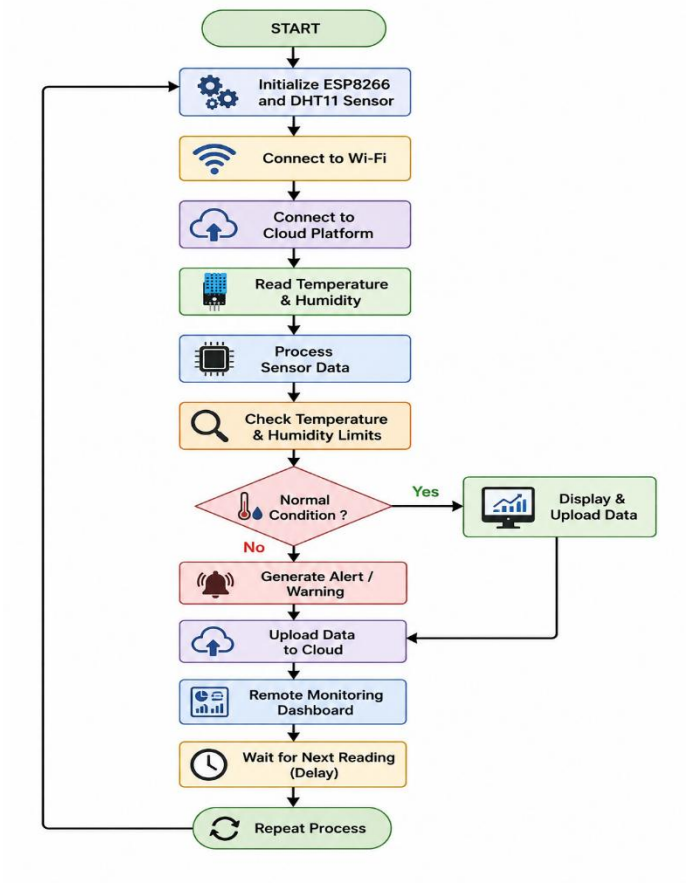
The proposed Smart Refrigerator Monitoring System uses a DHT11 temperature and humidity sensor and an ESP8266 NodeMCU to continuously monitor the environmental conditions inside a refrigerator. The DHT11 sensor measures the internal temperature and humidity, while the ESP8266 reads and processes the sensor data. The collected information is then transmitted to a cloud platform using Wi-Fi, allowing users to monitor refrigerator conditions remotely.

The system can identify abnormal conditions such as excessively high temperature or unusual humidity levels, which may indicate problems with the refrigeration system. The stored cloud data can be viewed through a dashboard and analyzed to observe environmental changes over time. This enables continuous monitoring and helps users maintain suitable conditions for food preservation.

The system operates as follows:

- Temperature Monitoring: Continuously measures the internal refrigerator temperature using the DHT11 sensor. The ESP8266 reads the temperature value at regular intervals.
- Humidity Monitoring: Measures the relative humidity inside the refrigerator and monitors changes in environmental conditions.
- Data Processing: The ESP8266 receives and processes the sensor readings and determines the current refrigerator condition.
- Wireless Communication: The ESP8266 uses Wi-Fi to transmit the collected data to the cloud platform.
- Cloud Data Storage: Temperature and humidity readings are stored on the cloud platform for remote monitoring and future analysis

4. System Architecture:

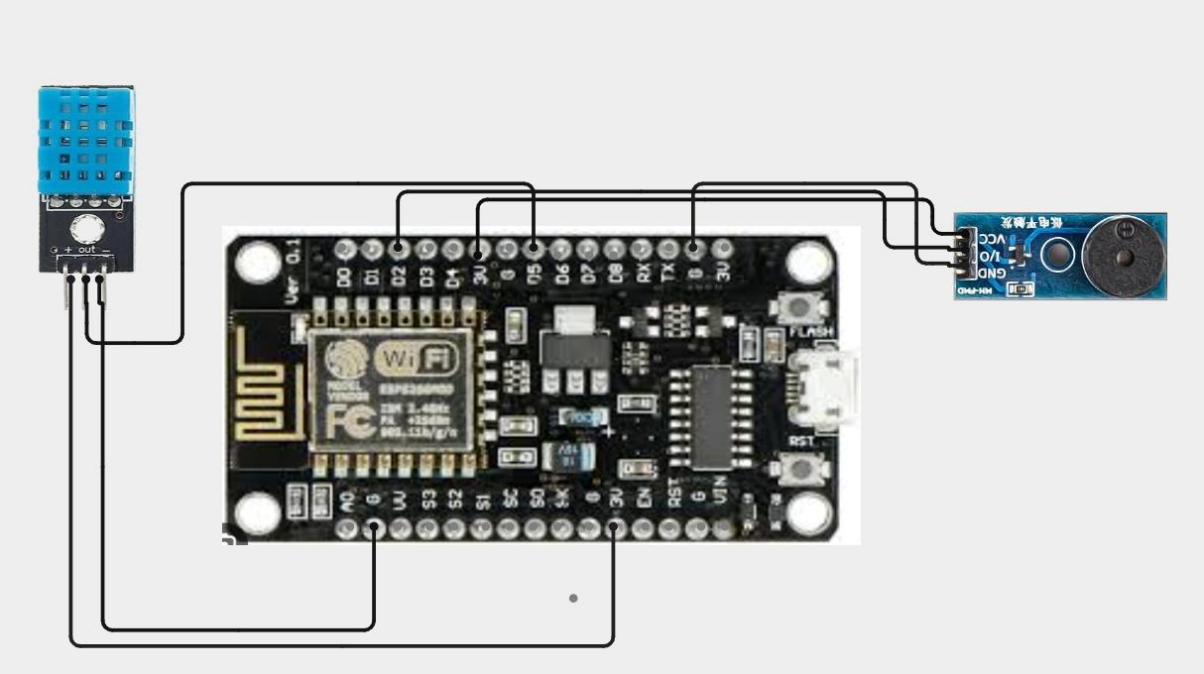


5. Circuit Table:

Circuit Table

Component	Connection
DHT11 Temperature & Humidity Sensor	Data → D4 (GPIO2)
OLED/LCD Display	SDA → D2 (GPIO4), SCL → D1 (GPIO5)
Buzzer	Signal → D5 (GPIO14)
Relay Module	IN → D6 (GPIO12)
ESP8266 NodeMCU	3.3V & GND → All Components
Wi-Fi Network	Wireless Connection
Cloud Platform	Receives Data from ESP8266

6. Circuit Design:



7. Algorithm:

Step 1: Start the system.
Step 2: Initialize the ESP8266 NodeMCU, DHT11 sensor, OLED/LCD display, buzzer, and relay module.
Step 3: Connect the ESP8266 to the configured Wi-Fi network.
Step 4: Establish communication with the cloud platform.
Step 5: Read temperature from the DHT11 sensor.
Step 6: Read humidity from the DHT11 sensor.
Step 7: Display the measured values on the OLED/LCD display.
Step 8: Compare the readings with predefined threshold values.
Step 9: If abnormal conditions are detected, activate the buzzer and relay.
Step 10: Upload temperature and humidity data to the cloud platform.
Step 11: Enable remote monitoring through the dashboard.
Step 12: Repeat the process at regular intervals.

8. Coding:

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>

//--------------- WiFi -----------------
const char* WIFI_SSID = "Luky003";
const char* WIFI_PASS = "kowshik03";

//----------- ThingSpeak ---------------
unsigned long CHANNEL_ID = 3459486;
const char* WRITE_API_KEY = "EHDR6D4T2FDB8LHZ";

WiFiClient client;

//------------ DHT11 -------------------
#define DHTPIN 14       // D5 = GPIO14
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

//------------ Buzzer ------------------
#define BUZZER_PIN 12   // D6 = GPIO12

void setup() {

  Serial.begin(115200);
  delay(1000);

  dht.begin();

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  // Connect WiFi
  Serial.print("Connecting to WiFi");

  WiFi.begin(WIFI_SSID, WIFI_PASS);
```

```
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  // Start ThingSpeak
  ThingSpeak.begin(client);

  Serial.println("ThingSpeak Ready!");
}

void loop() {

  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  // Check DHT
  if (isnan(temp) || isnan(hum)) {
    Serial.println("DHT Error");
    delay(2000);
    return;
  }

  String status;

  // Alert conditions
  if (temp > 30 || hum > 80) {

    status = "ALERT";
    digitalWrite(BUZZER_PIN, HIGH);

  }
  else if (temp > 20 || hum > 70) {

    status = "WARNING";
```

```
  digitalWrite(BUZZER_PIN, LOW);

}
else {

  status = "NORMAL";
  digitalWrite(BUZZER_PIN, LOW);
}

// Serial Monitor
Serial.println("------------------------");

Serial.print("Temperature: ");
Serial.print(temp);
Serial.println(" C");

Serial.print("Humidity: ");
Serial.print(hum);
Serial.println(" %");

Serial.print("Status: ");
Serial.println(status);

// ThingSpeak fields
ThingSpeak.setField(1, temp);
ThingSpeak.setField(2, hum);

// Status as numeric value
// 0 = NORMAL
// 1 = WARNING
// 2 = ALERT

if (status == "NORMAL") {
  ThingSpeak.setField(3, 0);
}
else if (status == "WARNING") {
  ThingSpeak.setField(3, 1);
}
else {
  ThingSpeak.setField(3, 2);
```

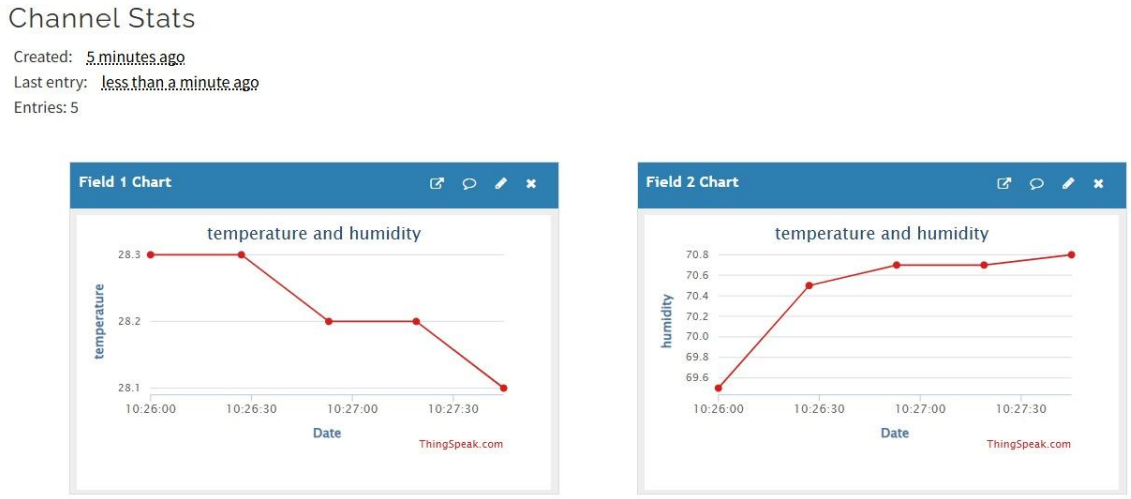
```
}

// Send data
int response = ThingSpeak.writeFields(CHANNEL_ID, WRITE_API_KEY);

if (response == 200) {
  Serial.println("ThingSpeak update successful!");
}
else {
  Serial.print("ThingSpeak update failed. Error: ");
  Serial.println(response);
}

delay(20000);   // ThingSpeak minimum update interval
}
```

9. System Working:



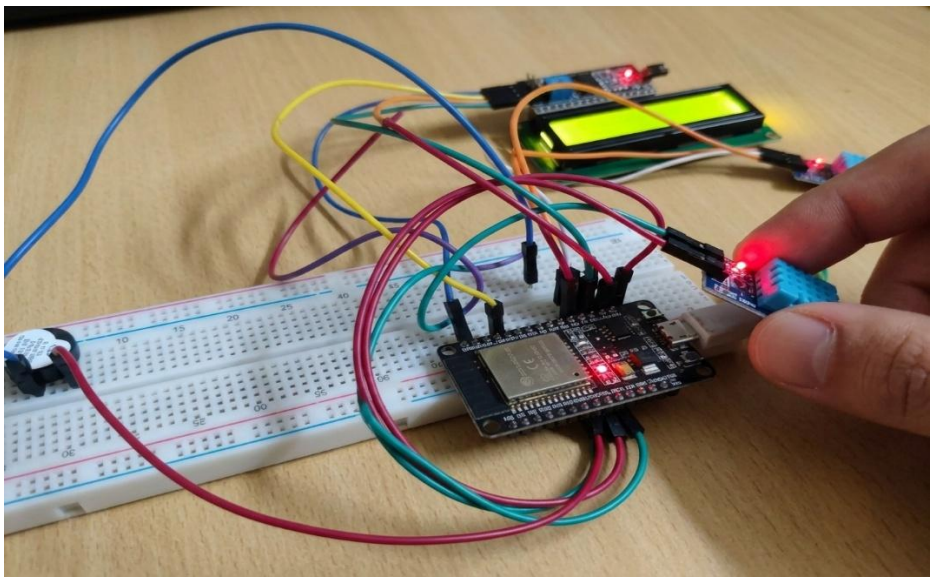
10. Working sequence:

Start → Initialize ESP8266 & DHT11 Sensor → Connect to Wi-Fi → Connect to Cloud Platform → Read Temperature & Humidity → Process Sensor Data → Check Threshold Limits → Display Values → Generate Alert (if required) → Upload Data to Cloud → Remote Monitoring Dashboard → Repeat Process

11. Bill of Materials:

S.No.	Component	Quantity
1	ESP8266 NodeMCU	1
2	DHT11 Temperature & Humidity Sensor	1
3	OLED/LCD Display	1
4	Relay Module	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	1 Set
8	USB Cable / Power Supply	1

12. Prototype Implementation:



13. Results and Performance Analysis:

Performance:

The system provides:

- Continuous temperature monitoring inside the refrigerator.
- Continuous humidity monitoring.
- Real-time sensor data acquisition.
- Wi-Fi based wireless communication.
- Cloud-based data storage.
- Remote monitoring through a dashboard.
- Historical data visualization and analysis.
- Detection of abnormal temperature and humidity conditions.

Advantages:

1. Continuous monitoring of refrigerator conditions.
2. Remote access through a cloud dashboard.
3. Early detection of refrigeration problems.
4. Cloud-based storage of sensor data.
5. Historical trend analysis of temperature and humidity.
6. Wireless communication using Wi-Fi.
7. Low-cost and easy-to-implement IoT solution.

Limitations:

1. The system depends on Wi-Fi and Internet availability.
2. DHT11 provides limited accuracy compared to industrial-grade sensors.
3. Sensor performance may be affected by environmental conditions.
4. Cloud monitoring is unavailable during network failures.
5. The prototype mainly monitors conditions and does not directly control refrigerator cooling.
6. Power interruptions can temporarily stop data collection and transmission.